Pre-course

Summary

The pre-course is a self-paced course that every trainee completes before the main course starts. It is gated by mastery: a trainee finishes only when they have passed every challenge, and no challenge can be skipped. The maximum is six weeks. Most trainees should take about four, and strong ones may take two.

Three pieces make it up. A GitHub repository holds the content and each trainee's work. SigmaBot, a fork of OpenCode that runs inside VS Code, teaches, coaches, grades and records progress. A thin dashboard over an Airtable database shows trainees their own progress and shows coaches everyone's. Trainees work on Sigma Labs laptops and get help through a shared Slack workspace.

Purpose And Constraints

Some trainees arrive from the application process without the technical skills to contribute to a team, and the first two weeks of the course go on closing those gaps. The current pre-course is self-paced, but we do not track progress well enough to know who is ready. The new pre-course replaces it as part of the tiered group model. It covers roughly the first four weeks of today's course, so a trainee who finishes it has working skills in the shell, Git, Python, APIs, SQL (Postgres) and testing.

The constraints come from the tiered group model:

- **Window:** each intake of about 20 trainees does the pre-course together in a fixed six-week window, then starts the course together.
- **Cap:** six weeks is a hard limit. A trainee who has not reached mastery by then leaves or is deferred.
- **Hiring:** hiring stays rolling, so a new hire may wait up to six weeks for the next window. Whether this fits Hire 2.0 is still to be confirmed with Jamie.
- **Launch:** the first cohort of the new model starts in February or March 2027.

Length is set by learning, not by hours. The pre-course is not sized as full-time work. Trainees fit it around other commitments and work at their own pace, and the content takes as long as mastery takes. Sigma Labs is high-trust here, as it is with the current pre-course, so there are no formal success measures.

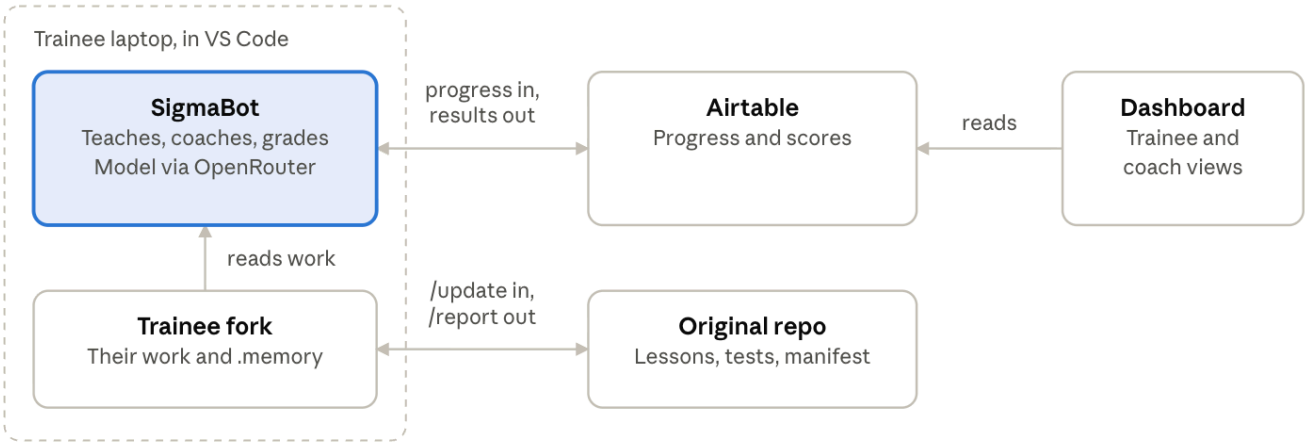
System Overview

A trainee spends nearly all their time in the repository and in SigmaBot. The dashboard sits on top as a progress view.

Piece	What it is	Who uses it
Content repository	A GitHub repository holding Markdown lessons, Python and SQL files with gaps, test suites and a manifest. Each trainee works in their own fork.	Trainees. Chris and Dan build it.
SigmaBot	A fork of OpenCode that runs in VS Code with access to the trainee's files. It teaches, coaches, asks questions, quizzes, reviews, accepts submissions and grades them.	Trainees
Dashboard	A thin web layer over an Airtable database that SigmaBot and the dashboard can both read and write. The trainee view shows their own progress and setup guidance. The coach view shows every trainee.	Trainees and coaches

At the start of every session SigmaBot reads the trainee's position from Airtable and tells them what to work on next. After each submission it writes the result back, and the dashboard reads the same tables. The manifest in the repository defines the course structure, and SigmaBot and the dashboard both use it.

SigmaBot links the trainee work to Airtable and the repo



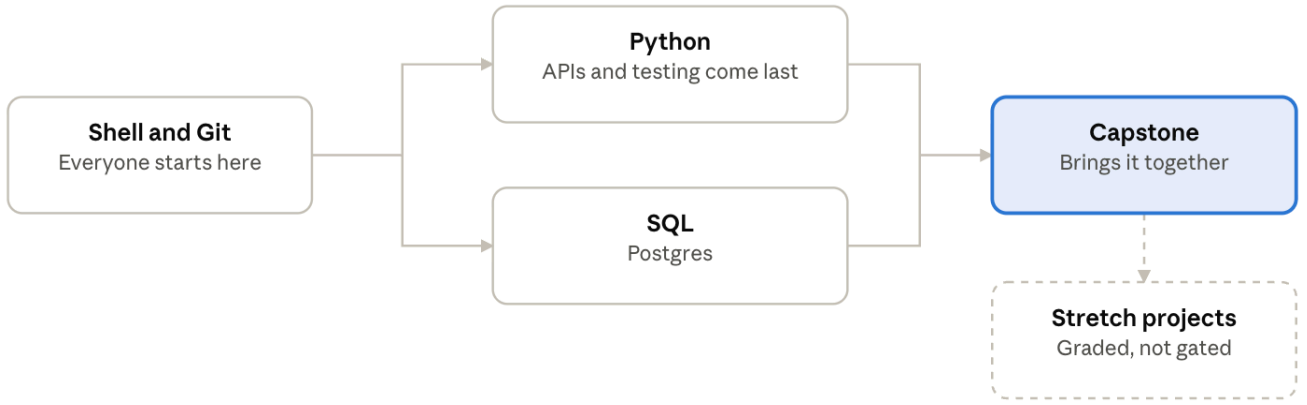
SigmaBot reads progress from Airtable and writes results back. The dashboard only reads Airtable. The trainee's fork pulls fixes from the original repository, and problem reports go to it.

Course Structure

Shell and Git come first and are fully gated. Python and SQL then run as two independent streams that a trainee can move between, and a capstone challenge at the end brings them together.

- **Shell and Git:** every trainee starts here. Nothing after it opens until it is complete.
- **Python:** an ordered list of challenges. APIs and testing are the later lessons within this stream, not modules of their own.
- **SQL:** an ordered list of challenges, using Postgres.
- **Capstone:** a larger challenge that uses the same flow and the same gates as every other challenge. It unlocks once shell and Git, Python and SQL are all complete.

Shell and Git first, then Python and SQL side by side, then the capstone



Python and SQL can be taken in either order or alternated. Stretch projects are optional and do not count towards finishing.

Everyone starts at the beginning, and there is no way to skip a challenge or test out of one. A challenge counts as complete only when the trainee has submitted work and passed. Challenges within a stream are taken in the order the manifest lists them. Early challenges are small and they grow in size. Later challenges reuse earlier skills, so Python work uses Git and the terminal as a matter of course. There is no separate refresher, and the main course keeps using these tools every day.

Where behaviours and Professional High Performance (PHP) sit in the pre-course is not yet decided. That space is kept open, and the question is listed under Open Questions.

Challenges

A challenge is a small project, not an isolated exercise. The trainee works through a few resources, then a scaffolded challenge framed around a stakeholder who wants an outcome. The stakeholder is a written profile in the Markdown. It is not an AI stakeholder, and the stakeholder interaction tool is not used in the pre-course. The instructions say clearly what to build, so the trainee does not have to draw requirements out of a conversation.

The framing is realistic. A SQL challenge gives a named stakeholder and a stated outcome and asks what query the trainee would write, instead of "write a query that does this". Challenges start small and grow, and capstone-style challenges check end-to-end understanding.

Each challenge folder holds the resources, the stakeholder profile and instructions, starter files with gaps, the visible tests, an encrypted non-functional rubric and guidance for generating the quiz.

How Teaching Works

Teaching comes from written Markdown in the repository and from curated outside resources, such as videos and articles that we know are useful. SigmaBot has internet access, so it can read those resources and the trainee can ask questions about them as they go. SigmaBot can also teach directly. Some Markdown steps can tell the trainee to paste a given command into SigmaBot to start a guided exchange.

AI-driven interactive workshops, listed on the Tiered Group Projects page as one way of measuring mastery, remain an option. How they would be implemented is undecided.

The number of challenges, their depth and their content follow the build schedule and are not specified here.

Submission, Grading And Mastery Gates

A trainee cannot move on from a challenge until it passes three separate gates. They can submit as many times as they need. There is no cap on attempts, and the only limit is the six-week window.

1. The trainee opens a challenge and starts working. At any point SigmaBot can give feedback, coaching and review.
2. When ready, they type `/submit` and SigmaBot asks what they want to submit.
3. SigmaBot runs the provided tests, runs the AI assessment against the hidden rubric, then gives a quiz. All of this runs on the trainee's laptop.
4. The trainee sees what failed, improves their work and submits again until every gate passes.

Gate	What it checks	How it is assessed	Pass mark
Functional	Whether the work does what the stakeholder asked	A test suite we provide. Trainees can see and run the tests.	90%
Non-functional	How the problem is solved, such as code quality and the efficiency of the SQL	AI assessment against a rubric held in an encrypted file shipped with the repository	90%
Understanding	Whether the trainee understands their own work and the course content	A quiz that SigmaBot generates from the submission and the course content	Set separately. Value to be decided.

The 90% applies to the functional and non-functional scores separately, so a trainee needs at least 90% on both. The quiz must also be passed before the submission completes.

Feedback names what is wrong. Passing on a first attempt is not the aim, and part of the skill is working out what to build instead of meeting a written requirement. Being told what failed and getting the chance to fix it is the mastery loop. After a failed quiz SigmaBot shows which questions were wrong and why, and each new attempt gets freshly generated questions.

Grading runs on the laptop, so no server sits between the trainee and the result. The AI-graded score can vary a little between runs, and Sigma Labs accepts that.

SigmaBot

SigmaBot is the trainee's teacher and coach. It is a fork of OpenCode, adapted to teach instead of code for the user. It runs inside VS Code, which gives it access to the trainee's files and lets trainees get feedback where they work. It also gives Sigma Labs control over AI use in the pre-course, which a general coding tool would not.

Default Behaviour

In its default mode SigmaBot teaches, coaches, asks questions, quizzes, reviews work and gives feedback. It can be Socratic. It can give code snippets and point to lines worth looking at. It never writes or fixes the trainee's code. This holds for the whole pre-course and does not loosen as trainees progress. Strict prompting enforces it.

Commands

Command	What it does	Can it edit files?
`/submit`	Asks what to submit, runs the tests, the AI assessment and the quiz, shows feedback and records the result in Airtable	No
`/update`	Pulls fixes from the original repository into the trainee's fork, carries completed work across and resolves merge conflicts	Yes, for this purpose only
`/it-support`	Fixes environment problems, such as Python not running on the laptop, by running commands and editing files	Yes, for this purpose only
`/report`	Asks the trainee about a problem with the content and files a GitHub issue	No

Editing is available only under the commands that need it. Every use of /update and /it-support is logged to Airtable so that coaches can see it on the dashboard.

Memory

SigmaBot keeps a `.memory` file at the repository root, written in plain Markdown. It holds what SigmaBot has learned about the trainee and adds context to every conversation. It is committed and pushed with the rest of the fork, so coaches can read it on GitHub and trainees can read it too. Nothing in it is hidden. Conversations themselves are not tracked.

AI Use

SigmaBot is the only AI trainees may use in the pre-course. Copilot is switched off on the laptops and other tools are discouraged. Sigma Labs cannot stop a trainee using another tool, so this relies on trust and on telling trainees clearly not to.

SigmaBot reaches its model through OpenRouter. The key is built into SigmaBot, and Sigma Labs fixes both the key and the model. A spending cap is set on the key in OpenRouter, and the key is rotated between intakes.

Dashboard And Coach View

The dashboard is a thin layer over the Airtable database. It has two views and sits behind a login.

Trainee view. A trainee sees only their own progress through the streams and which challenges are unlocked. The view also carries the setup instructions and guidance. There are no leaderboards, no comparison with other trainees and no competition.

Coach view. Coaches see every trainee: completion by challenge, attempt counts, time spent, time since the trainee started and when each submission happened. Conversations are not tracked.

Coaches judge progress themselves. There are no pace targets, and nothing pushes a trainee to hurry, because part of the test is whether they can finish in the time. Coaches step in to help when they see someone struggling.

SigmaBot reads from and writes to the same Airtable tables, so the dashboard and SigmaBot always show the same position.

Repo, Manifest And Updates

There is one original content repository. Each trainee forks it, works in their fork and commits there. SigmaBot reads their work from the fork.

Manifest

A versioned manifest file in the repository describes the course structure. The dashboard uses the same manifest, and the two are kept up to date together. Each challenge has an entry giving its stream, its prerequisites, the location of its tests, its encrypted rubric and its quiz guidance.

Updates

When a fix is needed mid-intake, Sigma Labs messages trainees and they run `/update` in SigmaBot. SigmaBot pulls the change into the fork and resolves any conflicts. Changes will be rare. When a challenge changes, SigmaBot carries the trainee's completed work across, so passes are not lost.

Reporting Problems

A trainee who finds a broken test, an unclear instruction or an impossible challenge types `/report`. SigmaBot asks a few questions and files an issue on the original repository, not the trainee's fork, so the team sees every report in one place. SigmaBot files the issue with a token built into the client that can only create issues, so trainees need no access to the original repository.

Each issue carries a title and labels (the challenge ID, the stream and a type: broken test, unclear instructions, environment problem or other), the trainee's GitHub username, what happened against what they expected, the error or failing test output with the file path, and SigmaBot's view of whether the cause looks like a content fault or a trainee difficulty, with the attempt count. The trainee's code is left out by default.

Access, Security And Cost

The pre-course relies on trust more than lock-down, and the controls are sized to that.

Area	Position
Login	Access sits behind a login: a sigmalabs.co.uk Google account, or an accepted-user list hosted on Auth0. Applicants have no Sigma Labs account, so they use the accepted-user list.
Hardware	Sigma Labs sends each trainee a laptop, so everyone is on the same hardware.
Model access	An OpenRouter key is built into SigmaBot. Sigma Labs fixes the key and the model and pays all usage, including during the pilot.
Airtable access	SigmaBot reads and writes Airtable with a token built into the client. A trainee could in principle read other trainees' records, which the high-trust approach accepts.
Hidden rubrics	Non-functional rubrics are encrypted files in the repository that SigmaBot decrypts at grading time. This stops casual reading, but a determined trainee could get past it.
Grading integrity	Grading runs on the trainee's laptop, so a trainee could in principle change their own scores. Sigma Labs is not worried about this. A trainee who skips the work is easy to spot and will not succeed on the course, and trainees are told so throughout. There are no extra integrity checks.
Other AI tools	Copilot is switched off. Other tools are discouraged but not blocked.

Trainee Support And Setup

Setup. The preferred route is a very detailed setup guide of commands to copy and paste, published on the dashboard and followed on the laptop Sigma Labs sends. Because the hardware is the same for everyone, few problems are expected. Two fallbacks remain available: a scripted installer that sets everything up in one step, or an onboarding session run by an Assistant Coach at the start of the window.

Help while working. SigmaBot's Socratic coaching is the first line of support. All pre-course trainees are also in a shared Slack workspace where they can ask questions. Assistant Coaches moderate it and answer when needed. Ad hoc tutoring and ad hoc workshops are not part of the base model, but they are a lever Sigma Labs can pull if trainees struggle with the pre-course itself.

Problems with the content. Trainees use `/report`, described under Repo, Manifest And Updates.

Stretch Projects And Early Finishers

A trainee who finishes the core challenges early fills the rest of the window with open-ended projects, such as building a game. These can be semi-guided or unguided, and they push trainees into more advanced topics.

Stretch projects go through the same submission flow and are graded, but they are not gated. Completing one does not count towards finishing the pre-course. Each result is logged as a record of what the trainee achieved. Trainees who finish early stay with their intake and start the course with it.

End Of The Window

Nobody starts the course without mastery. A trainee who has passed every gate, including the capstone, is ready to join their intake for induction.

A trainee who has not reached mastery when the six weeks end has one of two outcomes. They leave the programme, or they are deferred to the next intake if the real cause was that they were too busy to work on it. A coach reviews each case and decides between the two.

Pre-course completion rates are monitored from the first cohort, and hiring volume is adjusted to match, because each intake needs about 20 people who pass. The six-week cap stays fixed. If the bar proves too high for the hiring pool, the mastery thresholds can be tuned instead. What carries from the pre-course into the main course has not been decided.

Piloting And Build Order

Pilot. The pre-course can be tested before the first intake by giving people free access, with Sigma Labs paying the usage. Two groups are in mind. Alumni who have finished the training can test usability, but they already know the material, so they cannot show whether the learning works. Applicants can use it before their technical interview. It is extra support for them, and it shows whether they are actually learning.

Build order. SigmaBot comes first, then the content, then the dashboard. Chris and Dan own the build and delivery of all three pieces. Timelines are covered in a separate document. The first cohort of the new model starts in February or March 2027, and the pre-course window opens six weeks before that.

Open Questions

These were raised in the discussion and deliberately left for later.

- **Behaviours and PHP:** where they sit in the pre-course, and whether the earlier proposal still applies that trainees must take part weekly in a Slack questions channel or fail the pre-course.
- **Quiz pass mark:** the value, which is set separately from the 90% gates.
- **Hire 2.0:** whether rolling hiring into six-week intakes fits, to be confirmed with Jamie.
- **Applicants:** what happens to pre-course work an applicant does before being hired, and whether their results count in the hiring decision.
- **Handover to the course:** what carries from the pre-course into the main course, such as using results when forming the first teams, and whether a coach or the system confirms completion.
- **Workshops:** how AI-driven interactive workshops would be implemented, if at all.
- **Content volume:** the number, depth and content of challenges, which follow the build schedule.
- Tiered Group Projects page****: it sets the quiz and code-check bar at 70%, while this design uses 90% for the functional and non-functional gates. The page needs updating once the final numbers are settled.